MongoDB and Apache Kafka for Event-Driven Data Applications

Course Overview

This one-day course focuses on operating, monitoring, and maintaining MongoDB and Apache Kafka environments used in event-driven data applications. Learners will review key operational concepts, identify health and performance signals, practice routine maintenance tasks, and troubleshoot common MongoDB, Kafka, and connector issues.

Target Audience

- Operations, platform, and support engineers responsible for MongoDB and Kafka environments
- Database administrators and data engineers who monitor data pipelines and integration flows
- Application support teams maintaining event-driven applications in production
- Technical leads defining operational standards for MongoDB-Kafka solutions

Prerequisites

- Basic familiarity with MongoDB collections, indexes, and query behavior
- Basic familiarity with Kafka topics, consumers, producers, offsets, and consumer groups
- Experience using command-line tools, logs, dashboards, or monitoring systems
- Recommended: Docker Desktop, VS Code, and access to a MongoDB/Kafka sandbox before class

Learning Outcomes

- Identify key MongoDB, Kafka, and Kafka Connect health indicators.
- Monitor database performance, query behavior, replication status, and storage growth.
- Monitor Kafka broker, topic, partition, consumer lag, and throughput metrics.
- Perform routine maintenance checks for indexes, retention, connector tasks, and pipeline reliability.
- Troubleshoot common operational issues such as slow queries, consumer lag, failed connector tasks, and replay scenarios.

- Build a practical operational checklist for ongoing MongoDB-Kafka monitoring and maintenance.

Day 1: Monitoring, Maintenance, and Operational Readiness

Module 1: Operational Architecture Review

- How MongoDB, Kafka, and Kafka Connect work together in production event-driven systems
- Operational responsibilities across database, broker, connector, and application layers
- Common maintenance windows, deployment patterns, and change control considerations
- Typical failure points: slow queries, broker pressure, connector failures, lag, and data drift

Module 2: MongoDB Monitoring and Maintenance

- Monitoring database, collection, index, and storage growth indicators
- Reading slow query signals and reviewing query plans at an operational level
- Maintaining indexes for changing workload patterns
- Checking replication health, backups, restore readiness, and basic capacity risks
- Operational logging and alerting practices for MongoDB environments

Hands-on Lab 1: Inspect MongoDB Health and Query Performance

- Review database and collection statistics in a sample environment.
- Identify a slow or inefficient query pattern.
- Review index usage and propose a maintenance action.
- Create a short MongoDB health-check checklist.

Module 3: Kafka Monitoring and Maintenance

- Monitoring brokers, topics, partitions, replicas, throughput, and disk usage
- Tracking consumer lag, offset movement, and consumer group behavior
- Reviewing retention, compaction, topic configuration, and data lifecycle settings
- Recognizing symptoms of broker pressure, stalled consumers, and uneven partitions
- Alerting practices for operational Kafka environments

Hands-on Lab 2: Diagnose Kafka Consumer Lag and Topic Health

- Inspect topics, partitions, offsets, and consumer group status.

- Identify a consumer lag scenario and discuss likely causes.
- Review topic retention settings and operational risks.
- Create a short Kafka health-check checklist.

Module 4: Kafka Connect and MongoDB Connector Operations

- Monitoring connector workers, tasks, status, and error states
- Reviewing source and sink connector configuration for maintainability
- Handling failed tasks, malformed records, retries, and dead-letter queue patterns
- Validating data movement from MongoDB to Kafka and Kafka to MongoDB
- Operational considerations for credentials, configuration changes, and version upgrades

Hands-on Lab 3: Troubleshoot a MongoDB-Kafka Connector Flow

- Review connector status and task-level health.
- Trace a sample event from MongoDB through Kafka to a sink collection.
- Diagnose a failed connector task or misconfiguration scenario.
- Document recommended monitoring alerts for the connector flow.

Final Activity: Operations Runbook and Maintenance Checklist

Learners consolidate the day's monitoring and maintenance practices into a practical operations runbook for a MongoDB-Kafka data pipeline. The runbook defines routine checks, alert signals, escalation paths, troubleshooting steps, and maintenance actions for keeping the pipeline healthy after deployment.

Runbook Requirements

- Define daily MongoDB checks for storage, index usage, slow queries, replication, and backups.
- Define daily Kafka checks for broker health, disk usage, topic configuration, consumer lag, and throughput.
- Define Kafka Connect checks for worker health, connector status, task failures, retries, and dead-letter queues.
- Document common failure scenarios and first-response troubleshooting steps.
- Produce a concise maintenance checklist that can be used after the course.

Required Tools and Environment

- MongoDB local container or MongoDB Atlas sandbox with sample workload
- Apache Kafka local container environment or managed Kafka sandbox

- Kafka Connect runtime and MongoDB Kafka Connector for operational demonstrations
- Access to logs, connector status endpoints, and sample monitoring outputs
- VS Code or preferred editor for runbook and checklist creation
- Git, Docker Desktop, and command-line terminal